

Tech Leaders · Class 10

Machine Learning Rain Predictor

Teach a computer to forecast from 26,000 hours of weather

Features & labels

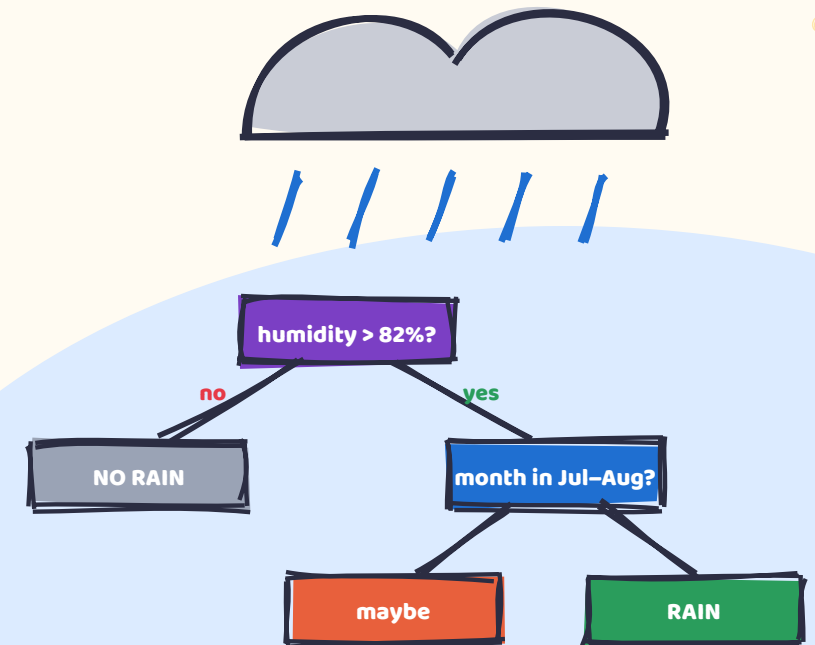
Decision trees

Honest testing

Live ESP32 data

Did you know?

Modern AI weather models like Google's GraphCast were trained on about 40 years of historical weather data.





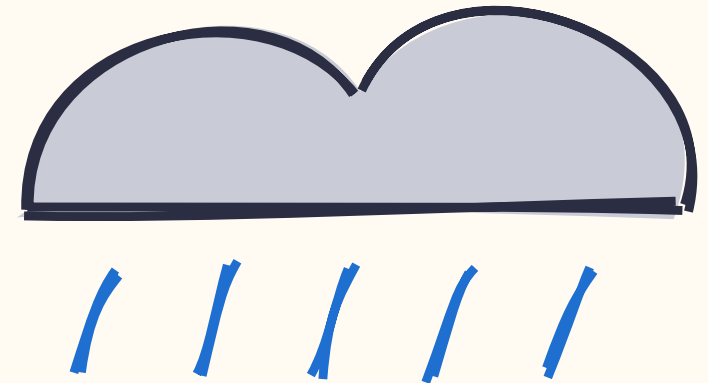
Let's think...

Should we take an umbrella tomorrow?



A forecaster's job

Your grandmother looks at the sky and says 'it will rain'. She is often right! She has seen **thousands** of days and learned the pattern. Can a computer learn the same way, from examples instead of rules?



Rules written by experts?

Pure guessing?

Learning from data!

Today: you train a **machine learning** model on real Islamabad weather history and run it on **live sensor data**.





Learning objectives

By the end of this class I can...



1



ML idea: explain how a model **learns from labelled examples** (features → label)

Proof: explain our features and label to a partner

2



Dataset: build a training table from **3 years of real Islamabad weather**

Proof: a DataFrame with features and a 'rain next hour' label

3



Train & test: train a **decision tree** and test it honestly against a **baseline**

Proof: our accuracy, recall and confusion matrix on 2024 data

4



Deploy: run the model on **live ESP32 sensor readings** and check it with a rain sensor

Proof: the laptop prints a live prediction every 5 s

5



Limits: explain why our model is weaker than a real forecast and how to improve it

Proof: two reasons and one improvement





Objective 1 • ML idea • video

Video time!



What is Machine Learning?

Oxford Sparks • about 2 minutes • youtu.be/f_uwKZIAeM0

Watch for...

- 1 How is machine learning different from writing **rules** by hand?
- 2 What does the computer need lots of?
- 3 Give one everyday example of ML from the video.



After the video: Write down one decision you make every day by 'learning from experience'. What examples taught you?





Objective 1 • ML idea

Features in, label out



The idea

Features: the facts we know now (temperature, humidity, month).

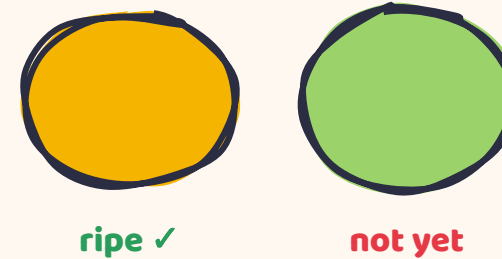
Label: what we want to predict (will it rain in the **next hour**? yes = 1, no = 0).

We show the model thousands of past rows where we already know the answer. It finds the pattern. Then we give it **new** features and it predicts the label.

Supervised

This is SUPERVISED learning: the training data comes with the correct answers.

Example: learning to spot ripe mangoes



Nobody gave you a rule. You squeezed hundreds of mangoes, smelled them, tasted them. Now 'soft + sweet smell' → ripe. Features: softness, smell, colour. Label: ripe or not.





Real world

Machine learning predicts all around us



Weather

AI models now rival traditional forecasts for 10-day weather



Farming

Predict crop disease from weather and leaf photos



Health

Flag patients at risk before symptoms get serious



Phones

Predict your next word as you type



In Pakistan, floods in 2022 affected 33 million people. How could better rain prediction help?

Early warning

Even a few hours of warning lets families move to safety.



Class game

Human decision tree

How to play

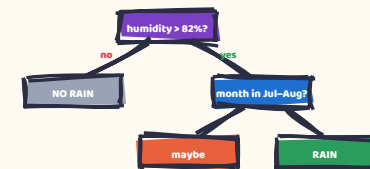
1. Teacher reads out 'weather cards': **humidity 90%, July, 31 °C...**
2. Teams write ONE yes/no question that best splits rain from no-rain cards.
3. Reveal the real answers. Which question split them most cleanly?
4. Add a second question on each branch. Score your tree on 10 new cards.

Then discuss

Was your best first question about humidity, temperature or month? Why?

Big idea

A decision tree is just the best yes/no questions, found automatically from data.





Objective 2 · Dataset

Real data: 3 years of Islamabad weather

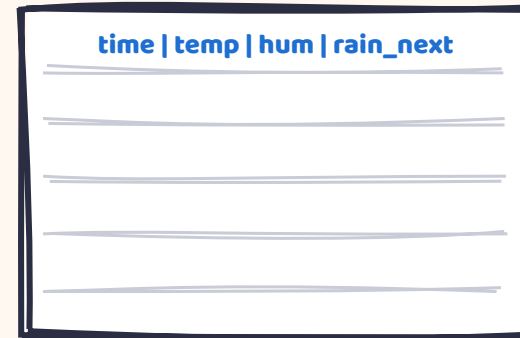


The idea

Open-Meteo gives free historical weather for any place, hour by hour. For Islamabad 2022–2024 that is about **26,000 rows**.

We build the label by looking one hour **ahead**: if precipitation next hour > 0.1 mm, **rain_next = 1**. This is called **shifting** the column.

Example: a diary



time	temp	hum	rain_next

If you wrote the weather in a diary every hour for 3 years, you could check: 'every time humidity was over 85% in July, did it rain an hour later?' That diary is our dataset.



Objective 2 · Dataset · Build step 1 of 4

Download and label the data

```
import requests, pandas as pd

url = ("https://archive-api.open-meteo.com/v1/archive"
      "?latitude=33.68&longitude=73.05"           # Islamabad
      "&start_date=2022-01-01&end_date=2024-12-31"
      "&hourly=temperature_2m,relative_humidity_2m,precipitation"
      "&timezone=Asia%2FKarachi")

df = pd.DataFrame(requests.get(url).json()["hourly"]).dropna()

df["time"] = pd.to_datetime(df.time)
df["month"] = df.time.dt.month
df["rain_next"] = (df.precipitation.shift(-1) > 0.1).astype(int)
df = df.iloc[:-1]           # last row has no 'next hour'

print(len(df), "rows")
print("It rained in the next hour", round(df.rain_next.mean()*100, 1), "% of the time")
df.to_csv("islamabad_weather.csv", index=False)
```

Look first!

Print **df.head()** and **df.describe()**. Do the numbers make sense?

Imbalanced

Rain happens in only a small % of hours. That will matter a lot when we test.

Save it

Saving the CSV means you don't download again every time.



Objective 3 · Train & test

Beat the lazy baseline



The idea

If it rains in about 5% of hours, a 'model' that **always says no rain** is **95% accurate**, and completely useless!

So we also measure **recall**: of all the hours it really rained, how many did we catch? And we test on **2024** data the model never saw, training only on 2022–2023. Testing on the past you trained on is cheating.

Metrics

Accuracy, recall, precision. Always ask: what is the cost of each kind of mistake?

Example: a smoke alarm

example · predicted: no / rain

actual	7820 TN	310 FP
	190 FN	440 TP

A smoke alarm that never beeps is 'right' 99.9% of days. But it misses every fire. We judge it by how many real fires it catches, and how often it beeps for burnt toast.





Objective 3 · Train & test · Build step 2 of 4

Train a decision tree



```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.metrics import accuracy_score, recall_score, confusion_matrix

df = pd.read_csv("islamabad_weather.csv", parse_dates=["time"])
features = ["temperature_2m", "relative_humidity_2m", "month"]
train = df[df.time < "2024-01-01"]      # learn from 2022-23
test  = df[df.time >= "2024-01-01"]    # exam on unseen 2024

model = DecisionTreeClassifier(max_depth=4, class_weight="balanced")
model.fit(train[features], train.rain_next)
pred = model.predict(test[features])

print("baseline:", round(1 - test.rain_next.mean(), 3))
print("accuracy:", round(accuracy_score(test.rain_next, pred), 3))
print("recall:  ", round(recall_score(test.rain_next, pred), 3))
print(confusion_matrix(test.rain_next, pred))
print(export_text(model, feature_names=features))
```

max_depth=4

At most 4 questions deep. Deeper trees memorise 2022 noise and do worse on 2024 (**overfitting**).

class_weight

'balanced' makes missing a rain hour count more, so the tree doesn't just say 'no' every time.

export_text

Prints the actual questions the tree learned. ML you can read!





Objective 3 · Train & test · read your results

Is our model any good?



Measure	Our number	Meaning
Baseline (always 'no rain')		high, but useless
Our accuracy		may be LOWER than baseline!
Our recall		how many rain hours we caught
False alarms (FP)		umbrella for nothing
Missed rain (FN)		got soaked

		example · predicted: no / rain	
actual		7820 TN	310 FP
		190 FN	440 TP



Don't panic

Lower accuracy with much higher recall can be the BETTER model. It depends what mistakes cost.



Objective 4 · Deploy · Build step 3 of 4

ESP32 sends live readings



```
#include "DHT.h"
DHT dht(4, DHT22);
const int RAIN_DO = 27;           // rain sensor digital out

void setup() {
  Serial.begin(115200);
  dht.begin();
  pinMode(RAIN_DO, INPUT);
}

void loop() {
  float t = dht.readTemperature();
  float h = dht.readHumidity();
  int wet = digitalRead(RAIN_DO) == LOW;  // LOW = water on pad
  if (!isnan(t) && !isnan(h)) {
    Serial.printf("%.1f,%.1f,%d\n", t, h, wet);
  }
  delay(5000);
}
```

Wiring

DHT22 data → GPIO 4. Rain sensor DO → GPIO 27, VCC → 3V3, GND → GND. Turn the blue knob to set wetness sensitivity.

Ground truth

The rain pad tells us if it is **actually** wet, so we can check the model live.

Test indoors

Drip a little water on the pad to simulate rain.



Objective 4 · Deploy · Build step 4 of 4

Live predictions



```
import serial, pandas as pd
from datetime import datetime
# ... train 'model' exactly as before, then:

ser = serial.Serial("COM5", 115200, timeout=10)
month = datetime.now().month
while True:
    line = ser.readline().decode(errors="ignore").strip()
    try:
        t, h, wet = line.split(",")
    except ValueError:
        continue
    X = pd.DataFrame([[float(t), float(h), month]],
                     columns=["temperature_2m", "relative_humidity_2m", "month"])
    p = model.predict_proba(X)[0][1] # chance of rain
    print(f"{t} °C {h} % rain chance {p:.0%} wet now: {wet}")
```

predict_proba

Gives a probability, like a real forecast: **70% chance of rain.**

Indoor trap

Indoor humidity and temperature are NOT the same as outdoors. Put the sensor by an open window, or the model is fed the wrong world.



Check

Breathe on the DHT22: humidity jumps. Does the predicted chance jump too?





Objective 5 · Limits

Why real forecasts beat ours

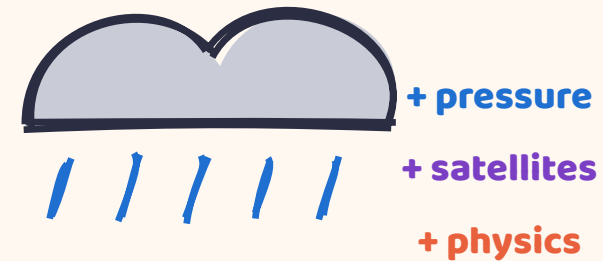


The idea

Our model sees only **3 numbers at one place**. Real forecasters use **air pressure** (falling pressure = storms coming), wind, satellite cloud images, radar and physics models of the whole atmosphere.

Also: our model learned from **weather station** data, but we feed it **a cheap sensor** in a classroom. Different data = worse predictions (called **data drift**).

Example: judging a cricket match from the score only



If you only know the score, you might guess the winner. If you also know the pitch, the weather, who is batting next and the run rate, you guess much better. More useful features = better predictions.





Test like an engineer

Results log



Experiment	Result	What we learned
Rows in dataset / % rain hours		
Baseline accuracy (2024)		
Our accuracy / recall (2024)		
max_depth 2 vs 4 vs 12: recall on 2024		
Live: chance when pad is dry / wet		

Debug checklist

requests error? Check internet or use the CSV the teacher shares.
ValueError in fit? A column has text or nan: print df.dtypes.



Change ONE thing at a time (depth, features, threshold) and write the result down. That is how ML engineers work.





Think about it

Who is to blame for a wrong forecast?



Scenario

A farming app uses an ML model to tell farmers 'no rain, safe to spray pesticide'. It rains. The crop is ruined and the river is polluted. The model was 92% accurate.

Discuss

- Who is responsible: the app, the data, the farmer?
- Should the app show **probabilities** instead of yes/no?
- Should people be told how often it is wrong?

Good practice

Publish how the model was tested, what it is bad at, and who it might fail. This is called a 'model card'.





Level up!

ML challenges



Choose your challenge



Pressure: add surface_pressure to the download and features. Does recall improve?



Trend: add humidity change over the last 3 hours (diff(3)) as a feature.



Other models: try RandomForestClassifier. Better on 2024?



On the ESP32: copy the export_text rules into if/else code so the ESP32 predicts with no laptop.

Think like an engineer

Why must we never test on the same years we trained on?



Boss level

Build a 'model card' web page for your predictor: data, metrics, limitations.





Objective check

Can I really do it? Prove it.



01 What are our features and our label? Why is the label shifted by one hour?



02 How many rows, and what % were rain hours?



03 Compare your model with the baseline. Which metric shows the model is useful?



04 Show a live prediction. What happens when you breathe on the sensor?



05 Give two reasons our model is weaker than the Met Office forecast.



Rate yourself

Colour one circle each. Red: not yet. Amber: with help. Green: I can explain it to a friend.

Teacher: Forecast challenge: each team predicts tomorrow's rain chance. Check the real weather next lesson!





Show what you know

What did we learn?

- ✓ ML learns patterns from labelled examples: features → label
- ✓ Shift the target to predict the future, not the present
- ✓ Always compare with a baseline and test on unseen data
- ✓ Recall matters when the thing you predict is rare
- ✓ More useful features and matching data beat a fancier model

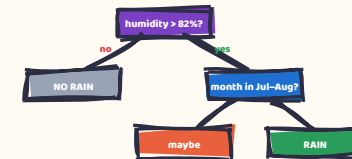


Exit ticket: a model is 95% accurate but has 0% recall. What is it doing?



At home

Check tomorrow's forecast on a weather app. Does it show a % chance? Why do they use probabilities?



Next class: AI
Fake News
Detector!

